

Fine-Tuning vs. In-Context Learning for Political Bias Classification

Eitan Derdiger | Yedidya Levine | Shira Yogev

Advanced Models of Language Understanding - Spring 2026

Abstract

We compare two ways to use language models for five-class political bias classification: LoRA fine-tuning and in-context learning (ICL). Qwen2.5-0.5B-Instruct was tested with both methods, while the larger Gemma 2 2B model was tested with ICL. We also used a majority baseline and TF-IDF with logistic regression. The best result came from Qwen fine-tuned on the full training set, with 0.826 accuracy and 0.711 Macro-F1. The TF-IDF baseline reached 0.607 Macro-F1, while the best ICL result reached 0.173. Fine-tuning needed enough labeled data before it became effective. ICL did not improve steadily when more examples were added, and its results also changed depending on which examples were used. In our setup, adapting a smaller model to the task worked much better than using a larger model only through prompting.

1. Introduction

Fine-tuning and in-context learning are two common ways to adapt a language model to a specific task. Fine-tuning updates model parameters using labeled examples. ICL keeps the model unchanged and describes the task through the prompt, sometimes with a few labeled examples. We wanted to compare these two approaches on the same political bias classification task and see where each method worked well and where it failed.

We used six research questions:

- RQ1.** How does LoRA fine-tuning compare with ICL for political bias classification?
- RQ2.** How does fine-tuning performance change as the amount of labeled training data increases?
- RQ3.** Does increasing the number of ICL demonstrations consistently improve performance?
- RQ4.** How sensitive is ICL to the specific demonstrations used in the prompt?
- RQ5.** Can a larger model using ICL outperform a smaller model adapted through fine-tuning?
- RQ6.** Which classes and error patterns explain the main differences between the approaches?

2. Related Work

Brown et al. [1] showed that large language models can perform new tasks from instructions and a small number of examples in the prompt. This is the main idea behind ICL. Hu et al. [2] introduced LoRA, a parameter-efficient fine-tuning method that keeps the original model weights frozen and trains small low-rank updates. These two methods match the main comparison in our project: learning from the prompt versus learning from labeled training data.

3. Data and Methods

3.1 Dataset and preprocessing

We used the Political Bias dataset from Kaggle. Each input contains the article title and article text. The target is one of five labels: left, lean left, center, lean right, or right. The original file contained 6,132 rows. We removed rows with missing or empty fields, failed article downloads, and duplicate title-text pairs. After cleaning, 4,600 articles remained.

Split	Total	left	lean left	center	lean right	right
Train	3,220	1,707	380	249	272	612
Validation	690	366	81	54	58	131
Test	690	366	82	53	58	131

Table 1. Stratified 70/15/15 split. The test set is imbalanced: 53.0% of its examples are labeled left.

The split used random seed 42 and kept the class distribution similar in train, validation, and test. Because more than half of the test set is left, a model that always predicts left already gets 53.0% accuracy. For this reason, Macro-F1 is our main metric and accuracy is secondary. Macro-F1 gives the same weight to every class instead of letting the largest class dominate the score.

3.2 Baselines and LoRA fine-tuning

We used two baselines: a majority classifier and TF-IDF with 5,000 features plus logistic regression. For fine-tuning, we used Qwen2.5-0.5B-Instruct with LoRA. Qwen was small enough to fine-tune with our available GPUs and could also be tested with ICL, which allowed a direct comparison.

We fine-tuned Qwen on nested subsets of the 3,220-example training set: 1% (32 examples), 5% (161), 10% (322), 25% (805), 50% (1,610), and 100% (3,220). LoRA used rank 8, alpha 32, dropout 0.05, and targeted the q, k, v, o, gate, up, and down projection modules. Training used learning rate $2e-4$, batch size 2, gradient accumulation 4, maximum length 2,048, and up to 10 epochs. We used validation loss, early stopping with patience 2, and restored the best checkpoint.

3.3 In-context learning

For ICL we tested Qwen2.5-0.5B-Instruct and Gemma 2 2B with 0, 1, 3, 5, and 10 demonstrations. Gemma was the larger ICL-only comparison, used to test whether model size could make up for not fine-tuning. For every nonzero-shot setting, we used three demonstration sets. Each full 10-example set had two examples from every class. The shorter prompts used prefixes of these sets, so they were not always class-balanced.

ICL prompts were limited to 4,096 tokens and generation used greedy decoding with at most 10 new tokens. A keyword parser mapped the generated text to a label. For nonzero-shot settings, the three predictions were combined with Python's mode rule; when there was no unique most common label, the implementation kept the first label it encountered. We report accuracy, Macro-F1, and invalid-output rate. Because runtime was collected on both L4 and T4 GPUs, it is not used as a main comparison.

3.4 Experimental comparisons

Experiment 1 compares the six fine-tuning data sizes with Qwen and Gemma ICL across the five shot counts. Experiment 2 compares full-data Qwen 0.5B LoRA with Gemma 2B ICL. We also use confusion matrices and per-class F1 scores to study the main errors.

4. Results

4.1 Baselines and fine-tuning as training data increases

Method	Setting	Accuracy	Macro-F1	Invalid
Majority baseline	100% train	0.530	0.139	0.0%
TF-IDF + logistic regression	100% train	0.755	0.607	0.0%
Qwen LoRA FT	1% train	0.530	0.147	0.1%
Qwen LoRA FT	5% train	0.530	0.139	0.0%
Qwen LoRA FT	10% train	0.519	0.203	0.1%
Qwen LoRA FT	25% train	0.616	0.305	0.0%
Qwen LoRA FT	50% train	0.668	0.452	0.0%
Qwen LoRA FT	100% train	0.826	0.711	0.0%

Table 2. Baseline and LoRA fine-tuning results on the final 690-article test set.

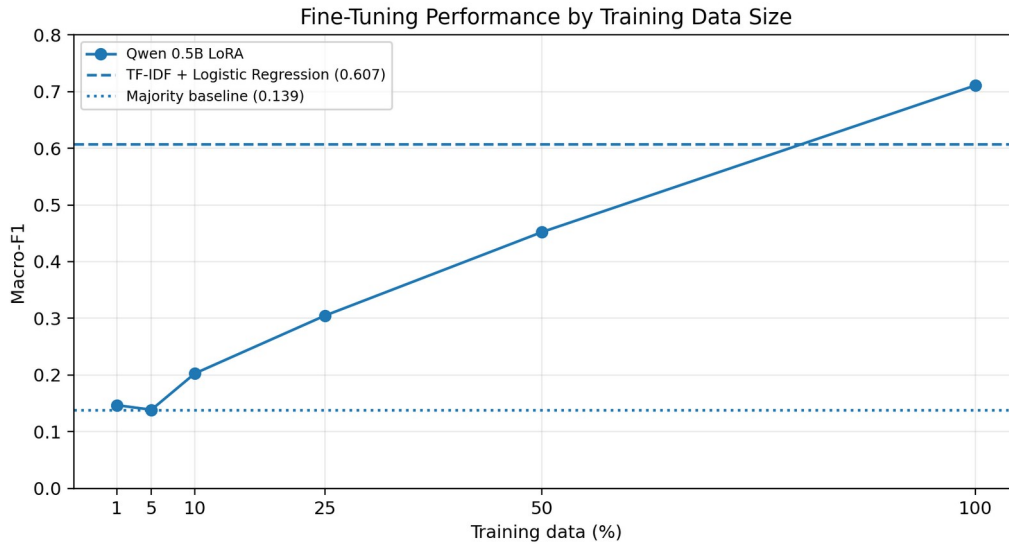


Figure 1. Macro-F1 as the amount of fine-tuning data increases. The two horizontal lines show the TF-IDF and majority baselines.

Direct answer to RQ1. Once enough labeled data was available, fine-tuning was much better than ICL. Full-data Qwen reached 0.711 Macro-F1, while the best ICL result was 0.173. With only 1% or 5% of the training data, however, fine-tuning stayed close to the majority baseline.

Direct answer to RQ2. After the very small training sizes, performance improved clearly as more data was used. Macro-F1 increased from 0.203 at 10% to 0.305 at 25%, 0.452 at 50%, and 0.711 at 100%. The biggest jump was from 50% to 100%, so the learning curve had not flattened out by the largest training size. The TF-IDF baseline was still better than every fine-tuned version up to 50%.

Accuracy alone hides part of this pattern. At 10% training data, accuracy was still below the majority baseline, but Macro-F1 improved because the model had started predicting minority classes. At 5%, it predicted left for 689 of the 690 test articles.

4.2 In-context learning: shot count and prompt sensitivity

Model	Shots	Accuracy	Macro-F1	Invalid
Qwen 0.5B	0	0.212	0.127	0.6%
Qwen 0.5B	1	0.122	0.045	0.0%
Qwen 0.5B	3	0.274	0.148	0.0%
Qwen 0.5B	5	0.210	0.110	0.0%
Qwen 0.5B	10	0.123	0.098	0.0%
Gemma 2B	0	0.181	0.159	3.2%
Gemma 2B	1	0.203	0.128	1.2%
Gemma 2B	3	0.241	0.153	0.3%
Gemma 2B	5	0.243	0.150	0.1%
Gemma 2B	10	0.235	0.173	0.3%

Table 3. ICL results after combining the three nonzero-shot prompt sets with the mode-based aggregation used in our code.

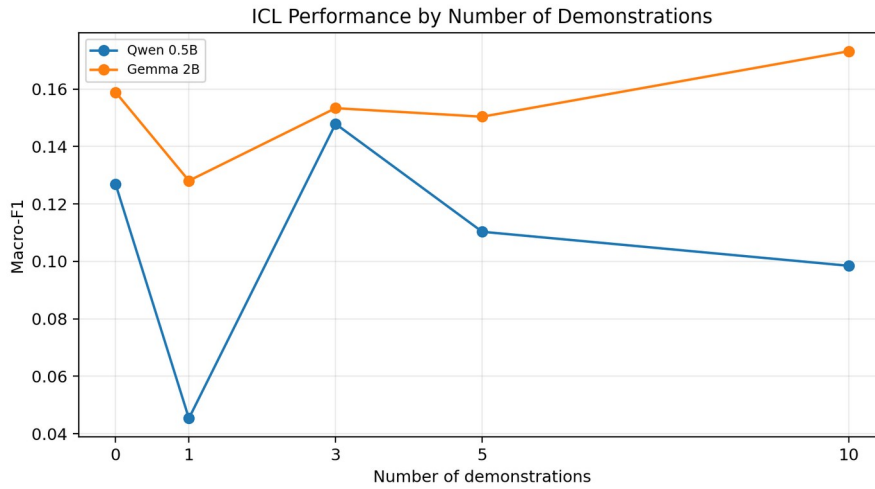


Figure 2. Macro-F1 versus the number of ICL demonstrations. More demonstrations did not lead to steady improvement.

Direct answer to RQ3. No. Qwen was best at 3-shot with 0.148 Macro-F1, but dropped to 0.045 at 1-shot and 0.098 at 10-shot. Gemma was best at 10-shot with 0.173, but its zero-shot result of 0.159 was still better than its 1-, 3-, and 5-shot results. In our experiments, adding more demonstrations was not a reliable way to improve ICL.

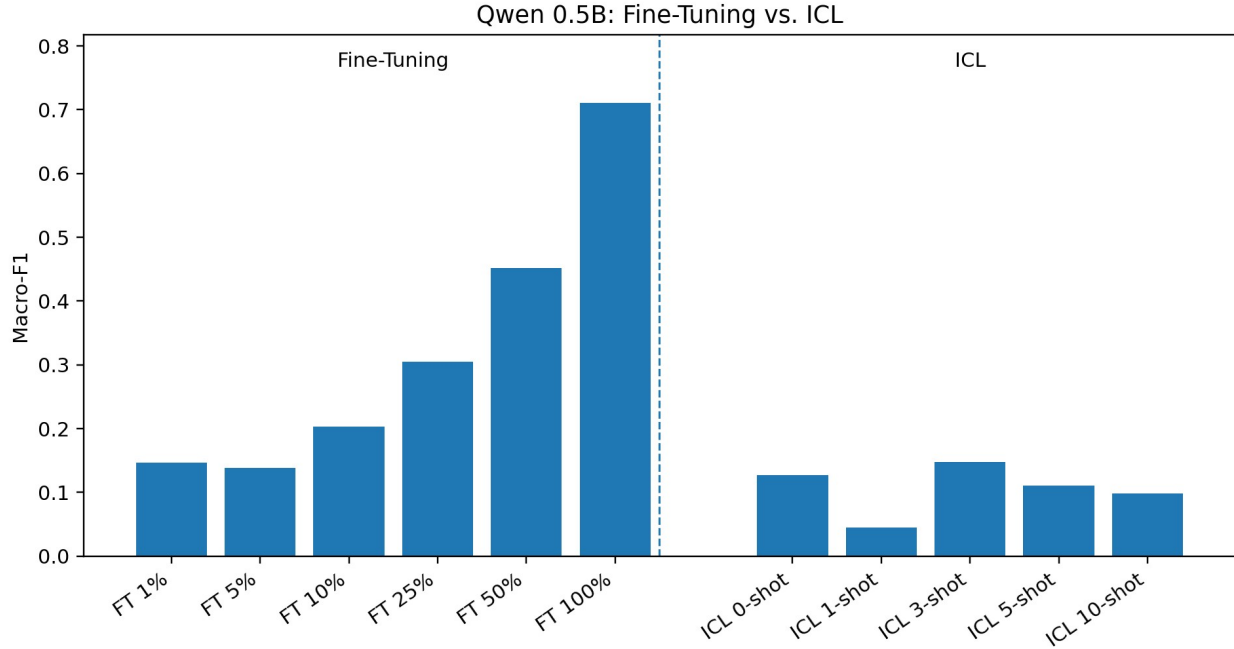
Condition	Set 1	Set 2	Set 3
Qwen 1-shot	0.169	0.044	0.044
Qwen 5-shot	0.172	0.043	0.112
Gemma 10-shot	0.207	0.147	0.180

Table 4. Selected Macro-F1 scores before aggregation. The same model and shot count can change a lot when the demonstrations change.

Direct answer to RQ4. Yes. Qwen was very sensitive to the chosen examples. Its 1-shot sets scored 0.169, 0.044, and 0.044 Macro-F1, while its 5-shot sets scored 0.172, 0.043, and 0.112. Gemma also changed across sets, but less sharply in the examples we checked. Strictly invalid outputs were uncommon, but the parser could still turn a response that contained more than one label into a valid class.

4.3 Direct comparison: Fine-tuning vs. ICL on Qwen

Figure 3. Direct comparison of Qwen 0.5B under LoRA fine-tuning and ICL. Fine-tuning surpassed all Qwen ICL settings by 10% training data and widened the gap as more labeled data was added.



Direct answer to RQ1 on the same model. When we compare the same Qwen 0.5B model under the two adaptation methods, fine-tuning clearly does better once a modest amount of labeled data is available. The best Qwen ICL result was 0.148 Macro-F1 at 3-shot. Fine-tuning reached 0.203 with 10% of the training data, 0.305 with 25%, 0.452 with 50%, and 0.711 with the full training set.

This direct comparison is important because it separates the effect of adaptation method from the effect of model size. In our setup, the main gain came from task-specific adaptation, not from keeping the model fixed and adding demonstrations in the prompt.

4.4 Small fine-tuned model versus larger ICL model

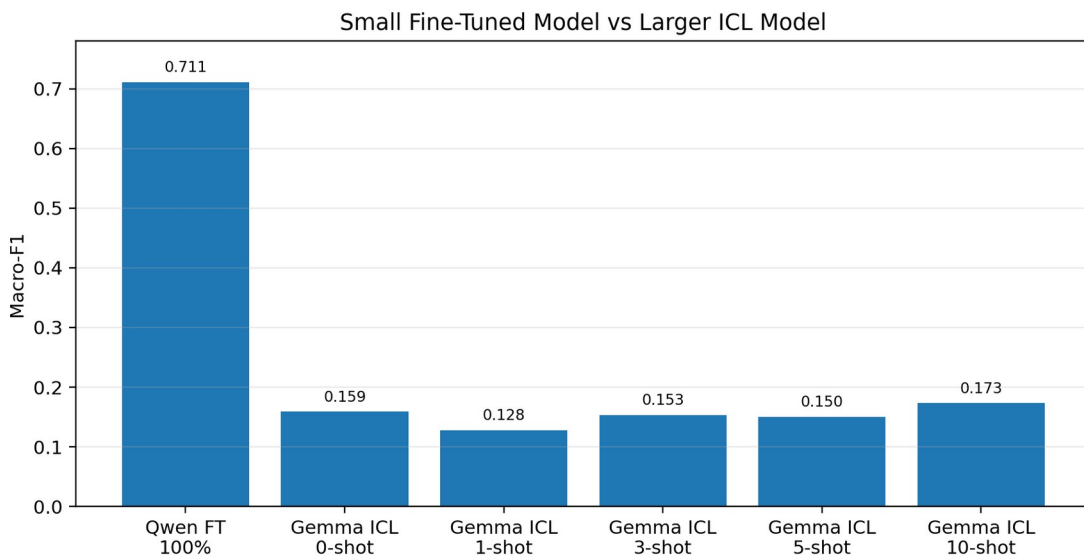


Figure 4. Full-data Qwen 0.5B with LoRA compared with Gemma 2B under ICL. The best Gemma result was 0.173 Macro-F1, compared with 0.711 for fine-tuned Qwen.

Direct answer to RQ5. No. The larger Gemma 2B model reached at most 0.173 Macro-F1 with ICL, compared with 0.711 for the smaller Qwen 0.5B after full-data LoRA fine-tuning. Gemma did beat Qwen when both models used ICL at the same shot count, so the larger model helped somewhat. However, the gain from task-specific fine-tuning was much larger.

4.5 Class-level error analysis

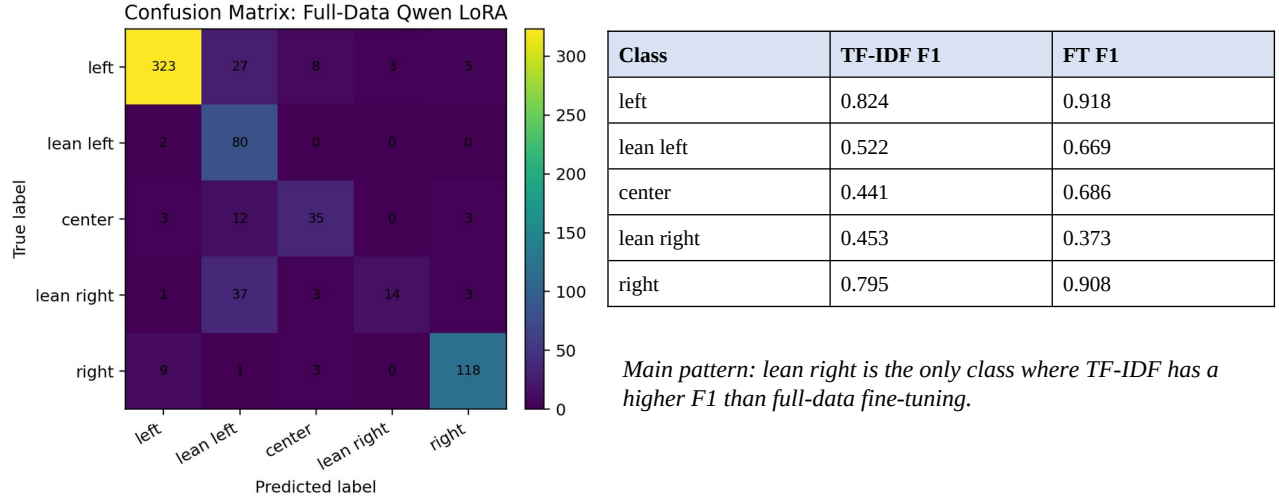


Figure 5 and Table 5. Confusion matrix for full-data Qwen LoRA and per-class F1 compared with TF-IDF + logistic regression.

Direct answer to RQ6. Lean right was the hardest class for the best fine-tuned model. Only 14 of 58 lean-right articles were classified correctly, while 37 were predicted as lean left. Full-data Qwen had a higher F1 than TF-IDF in four of the five classes; lean right was the only exception. Center is similarly rare but reached 0.686 F1, so class frequency by itself does not explain the problem.

5. Discussion

The clearest pattern is that fine-tuning needed enough labeled data before it became useful. With very small training sets, Qwen mostly fell back to the majority class, and the simple TF-IDF baseline was better than every fine-tuned model up to the 50% setting. With the full training set, Qwen improved strongly and passed both baselines by a clear margin.

ICL behaved differently. More demonstrations did not consistently help, and changing the examples in the prompt sometimes changed the score a lot. Gemma usually did better than Qwen when both used ICL, so using a larger model helped to some degree. Even so, Gemma stayed far behind the smaller Qwen model after fine-tuning. Our result is therefore about this task and this setup; it is not a claim that fine-tuning is always better than ICL.

The strong TF-IDF baseline is also important. The cleaned dataset has 22 publishers. Every publisher is linked to only one bias label, and all 22 publishers in the test set also appear in the training set. This means our split tests new articles from publishers the models have already seen, not completely unseen publishers. The models may therefore learn writing style or publisher-specific words in addition to political content. This may be one reason why TF-IDF works so well.

6. Limitations and Future Work

The biggest limitation is the overlap of publishers between train and test. A stronger follow-up would keep complete publishers out of training and use them only for testing. ICL could also be tested with model-specific chat templates, balanced demonstration sets at every shot count, and constrained generation that only allows one label.

Our parser searches the generated text for label phrases in a fixed order. If a model writes more than one label, the parser can choose a different label from the first one written. We kept the results of the implemented pipeline, but this means the reported ICL scores should not be treated as the best possible ICL performance. The mode-based aggregation also always breaks ties in the same way when all three prompt sets disagree.

Runtime was measured on both NVIDIA L4 and T4 sessions, so we do not make claims from the cross-method timing comparison. Future work could also repeat the experiments with more random seeds, test larger instruction-tuned models, and try a hybrid method that combines the strong lexical baseline with the fine-tuned model.

7. Conclusion

For our five-class political bias task, LoRA fine-tuning was the best approach once enough labeled data was available. Full-data Qwen reached 0.711 Macro-F1, compared with 0.607 for TF-IDF + logistic regression and 0.173 for the best Gemma ICL setting. We also found that ICL did not improve steadily as more demonstrations were added and could change noticeably when the examples changed. The practical result from our experiments is that a smaller model that is trained for the task can outperform a larger model that only receives examples in its prompt.

8. Contribution Statement

All three authors contributed equally to the project. We worked together in shared Zoom sessions throughout the project. We jointly chose the research questions, planned the experiments, wrote and reviewed the code, ran the models, compared the methods, analyzed the results, and prepared the final report.

References

- [1] T. B. Brown et al., "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems* 33, 2020.
- [2] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *International Conference on Learning Representations*, 2022.